

Full Stack Developer Intern Live Dashboard Task

Hi,

Thank you for applying for the **Full Stack Developer Internship at Instant Mechanic**.

We have received a large number of applications, and this practical assignment is the next step in our selection process.

We want to see how you approach a **real-world full-stack product**, including frontend, backend, APIs, database, deployment, UI/UX, and problem-solving.



Assignment: Build a Live Operations Dashboard

Build a modern **Live Vehicle Service Operations Dashboard** for a company like Instant Mechanic.

Imagine that this dashboard will be used by an operations team every day to monitor bookings, mechanics, customers and revenue.

The final product should feel like a **real production SaaS dashboard**, not a basic coding assignment.



1. Frontend

You can use any modern frontend technology.

Recommended:

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui

Deploy the frontend on **Vercel**.

The dashboard should include:

Overview

- Total bookings
- Today's bookings
- Completed bookings
- Pending bookings
- Cancelled bookings
- Total revenue
- Active mechanics
- New customers

Analytics

Include visually appealing charts for:

- Bookings over time
- Revenue over time
- Booking status
- Service/category breakdown

Bookings

Create a professional booking table with:

- Booking ID
- Customer
- Vehicle
- Service
- Mechanic
- Status
- Amount
- Date/time

Include useful features such as:

- Search
- Filtering
- Sorting
- Pagination

Mechanics

Display:

- Mechanic name
- Current status
- Jobs completed
- Current/last booking

2. Backend

Build a proper backend API.

You may use any technology you are comfortable with, for example:

- Node.js / Express
- Node.js / NestJS
- Python / FastAPI
- Python / Django
- Java / Spring Boot
- Or another suitable technology

The backend should expose APIs similar to:

GET /api/dashboard

GET /api/bookings

GET /api/bookings/:id

GET /api/mechanics

GET /api/customers

You are free to design the API structure yourself.

We will evaluate the quality of your API design rather than whether you use these exact endpoints.

3. Database

Use a proper database.

For example:

- PostgreSQL
- MySQL
- MongoDB

The application should actually retrieve data from the backend/database.

Do not build the entire dashboard using hardcoded frontend JSON.

Create realistic seed/sample data.

For example:

- 500+ bookings
 - 50+ customers
 - 20+ mechanics
 - Multiple service categories
 - Different booking statuses
 - Different dates and amounts
-



4. Live Dashboard

The dashboard should behave like a live operations system.

Implement one of the following:

Basic

Automatic API polling / refresh.

Better

Server-Sent Events.

Best

WebSockets / real-time updates.

For example, when a booking status changes:

Pending → Assigned → Mechanic On The Way → Completed

the dashboard should be capable of reflecting the change without requiring a complete page reload.



5. Deployment

The project must be publicly accessible.

Frontend

Deploy on:

Vercel

Backend

Deploy using:

AWS Free Tier

You may use services such as EC2 or other appropriate AWS services.

Code

Host the source code on:

GitHub

Your final submission should include:

GitHub:

Frontend:

Backend:

API Documentation:

All links must be accessible.



6. AI Usage — FULLY ALLOWED

You are encouraged to use AI.

There is **no restriction on using AI tools**.

You may use:

- ChatGPT
- Claude
- Gemini
- Cursor
- GitHub Copilot
- Windsurf
- Lovable
- v0
- Replit
- Other AI coding/design tools

You can use AI for:

- Writing code
- Designing UI
- Debugging
- Database design
- API design
- Documentation
- Testing
- Deployment
- Generating sample data
- Improving UX
- Writing SQL
- Architecture decisions

But there is one important rule:

You must understand what you submit.

During the next round, we may ask you to:

- Explain your architecture
- Explain important parts of your code
- Debug an issue
- Add a small feature
- Modify an API
- Make a database change
- Explain why you made certain technical decisions

Using AI is not a problem.

Using AI without understanding the result is.



7. Design & UX

This is intentionally open-ended.

Make the dashboard look impressive.

We will look at:

- Visual hierarchy
- Typography
- Spacing
- Responsiveness
- Charts
- Tables

- Navigation
- Loading states
- Error states
- Empty states
- Micro-interactions
- Overall polish

Think:

"If Instant Mechanic gave this dashboard to its operations team tomorrow, would it actually be pleasant to use?"

8. Bonus Features

These are optional but can significantly improve your submission:

- Authentication
- Role-based access
- Admin/Operations roles
- Dark mode
- Real-time notifications
- WebSockets
- Map showing mechanic locations
- Booking detail page
- Mechanic detail page
- Advanced filters
- Export CSV
- API rate limiting
- Swagger/OpenAPI
- Automated tests
- Docker
- CI/CD
- GitHub Actions
- Error monitoring
- Caching
- Optimized database queries
- Responsive mobile dashboard

You do **not** need to implement everything.

A smaller but extremely polished product is better than a huge unfinished application.



9. README

Your GitHub repository should contain a good README.

Include:

Project Overview

What you built and why.

Tech Stack

Frontend, backend, database, infrastructure, etc.

Architecture

Explain:

Frontend



API



Backend



Database

Local Setup

Explain how someone can run the project locally.

Environment Variables

Document required environment variables.

API Documentation

Explain your major endpoints.

Deployment

Explain how you deployed the frontend and backend.

AI Usage

Briefly mention:

- Which AI tools you used
 - What you used them for
 - Any significant code/features generated with AI
 - What you personally modified or implemented
-



10. What We Will Evaluate

We are not primarily testing how fast you can type code.

We will evaluate:

1. Product thinking

Can you turn an open-ended requirement into a useful product?

2. Frontend

Is the UI modern, responsive and usable?

3. Backend

Is the architecture clean and maintainable?

4. APIs

Are the APIs logically designed?

5. Database

Is the data model sensible?

6. Engineering

How do you handle errors, edge cases and performance?

7. Deployment

Can you actually deploy a working full-stack application?

8. Problem solving

How do you approach problems when things don't work?

9. AI usage

Can you use AI as an engineering multiplier rather than blindly copying generated code?

10. Attention to detail

Does the product feel finished?



Deadline

Please complete the assignment within **48 hours** of receiving it.

We understand that candidates may have college, work or other commitments, so **quality is more important than the number of features**.



Submission

Please submit:

1. Name
2. GitHub Repository
3. Live Vercel URL
4. Live Backend URL
5. API Documentation
6. Short architecture explanation
7. AI tools used
8. What you are most proud of in the project

Please make sure all links are publicly accessible.



Important

You are **not expected to build the perfect system**.

We are more interested in seeing:

**How you think → How you build → How you debug → How you use AI →
How you ship.**

Shortlisted candidates may be asked to make a small live change to their project during the interview.

Good luck! 🚀

Team Instant Mechanic